

26년 경북대 AX Intensive 교육

1주차 – pandas 기본



소 속 : 경북대학교/인공지능학과

이 름 : 김현철 교수

E-Mail : hyunchul_kim@knu.ac.kr

Timetable

시간표		학습 내용
1차시	09:30 – 10:00	OT (30분)
2차시	10:00 – 10:50	Pandas 기본 (이론/ 실습)
3차시	11:00 – 11:50	Pandas 기본 (실습)
Lunch Break (11:50 – 13:30)		
5차시	13:30 - 14:20	개인별 AX 과제 정의(발표) 및 랩실 멘토링
6차시	14:30 – 15:20	
7차시	15:30 – 16:20	
8차시	16:20 – 17:30	

1. Packages

- <https://numpy.org/>



- <https://pandas.pydata.org/>

표 형식 데이터의 개념

1. 표 형식 데이터의 개념

- 표 형식 데이터란?
 - 행(row)과 열(column)로 구성된 2차원 구조의 데이터
 - 각 행은 하나의 관측치(observation), 각 열은 하나의 변수(variable)를 나타냄
 - 일상생활 예시: 학생 성적표, 회사 직원 명단, 온라인 쇼핑몰 상품 목록
- 표 형식 데이터의 장점
 - 직관적이고 이해하기 쉬운 구조
 - 다양한 데이터 타입을 한 번에 저장 가능
 - 데이터 분석 도구들이 표준적으로 지원

1. 표 형식의 데이터 다루기

- 표의 주요 구성요소
 - 행 (row)
 - 열 (column)
 - 셀 (cell)
 - 헤더(레이블) (header, label)
 - 인덱스 (index)

1. 표의 행과 열을 나타내는 다양한 용어

- 행(row)
 - 관측치 (observed value)
 - 개체 (instance)
 - 기록 (record)
 - 경우 (case)
 - 사례 (example)
 - 멤버 (member)
- 열(column)
 - 특성 (feature)
 - 속성 (attribute)
 - 변수 (variable)
 - 필드 (field)

1. 데이터는 행 단위로 채워진다

	total_bill	tip	sex	smoker	day	time	size
1	27.20	4.00	Male	No	Thur	Lunch	4
2	20.27	2.83	Female	No	Thur	Lunch	2
3	16.00	2.00	Male	Yes	Thur	Lunch	2
4	13.16	2.75	Female	No	Thur	Lunch	2
5	16.82	4.00	Male	Yes	Sun	Dinner	2
6	23.17	6.50	Male	Yes	Sun	Dinner	4



7	18.15	3.50	Female	Yes	Sun	Dinner	3
---	-------	------	--------	-----	-----	--------	---

1. 데이터의 형식

- 범주형
 - 카테고리로 나눈 것
 - 명목형 / 순서형
- 수치형
 - 수량 그 자체를 의미
 - 연속형 / 이산형

	total_bill	tip	sex	smoker
1	27.20	4.00	Male	No
2	20.27	2.83	Female	No
3	16.00	2.00	Male	Yes
4	13.16	2.75	Female	No
5	16.82	4.00	Male	Yes
6	23.17	6.50	Male	Yes

1. 데이터 타입

Pandas	Python	사용
Object	str	텍스트 또는 혼합 숫자 숫자가 아닌 값
Int64	Int	정수형 숫자
Float64	Float	부동소수점(실수형) 숫자
Bool	Bool	True/False
Datetime64	NA	날짜 및 시간
Timedelta[na]	NA	두 날짜/시간 사이의 차이
category	NA	범주형 데이터

1. 데이터 공간

- 변수의 개수만큼 공간의 축(=차원)이 생성
- 각각의 관측치는 공간상의 한 점으로 표현
- 데이터 공간에는 관측치 개수 만큼의 점을 찍을 수 있음
- 시각화(visualization)를 통해 확인할 수 있음

pandas 기초

2. Pandas 소개

- 구조화된 데이터나 표 형식 데이터를 빠르고 쉽게 다루도록 설계
- 고수준의 자료구조와 데이터 분석 도구 포함
- NumPy의 배열 기반 계산 스타일 차용
 - 반복문 없이 데이터 처리가 가능한 점
- 이름의 유래 : **panel data + Python data analysis**

2. Pandas 소개

- pandas를 사용하는 이유
 - 엑셀보다 빠른 대용량 데이터 처리
 - 반복 작업의 자동화 가능
 - 다양한 데이터 형식 지원 (CSV, Excel, JSON, SQL 등)
 - 강력한 데이터 분석 및 시각화 기능
- pandas의 핵심 개념
 - 벡터화 연산: 반복문 없이 전체 데이터에 연산 적용
 - 인덱스 기반 데이터 접근
 - 결측치 처리의 용이성

2. pandas 설치

- **Installation**

```
pip install pandas
```

- colab 에는 설치되어 있음

2. 모듈 import

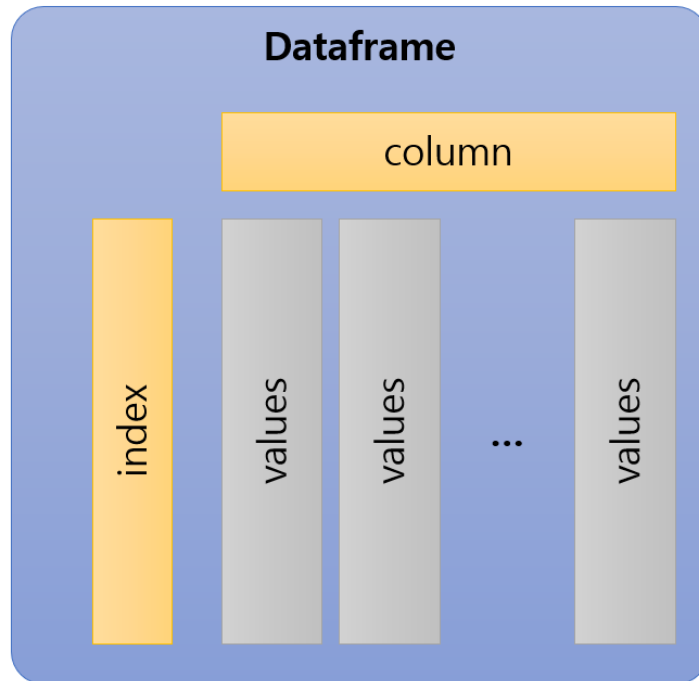
```
import numpy as np  
import pandas as pd  
pd.__version__
```

→ '2.0.3'

2.

핵심 객체

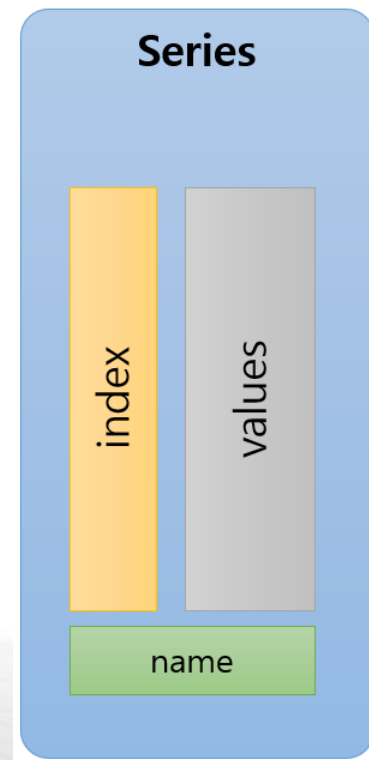
- Series
- Dataframe



Series 객체

3. Series

- Series란?
 - 1차원 배열 + 인덱스의 조합
 - 동일한 데이터 타입의 값들을 순서대로 저장
 - 각 값에 고유한 라벨(인덱스) 부여 가능
 - 단일 열에 대한 데이터를 저장하고 처리하는 데 사용
- Series의 특징
 - 인덱스를 통한 빠른 데이터 접근
 - 벡터화 연산 지원
 - 결측치(NaN) 처리 기능 내장
- 인덱스(Index)와 값(Value)으로 구성
 - 인덱스는 각 데이터 요소에 대한 고유한 식별자
 - 값은 해당 데이터 요소의 실제 값
 - Index – value 는 1:1로 대응



3. Series 만들기

- From

- List
- Dict
- Numpy array

- list

```
data = [5, 6, 7, 8]  
ser = pd.Series(data)
```

```
0    4  
1    7  
2   -5  
3    3  
dtype: int64
```

3. Series 만들기

- dict

```
data = {0: 5,  
        1: 6,  
        2: 7,  
        3: 8}  
ser = pd.Series(data)  
ser
```

```
0    5  
1    6  
2    7  
3    8  
dtype: int64
```

- numpy array

```
data = np.arange(4)+5  
ser = pd.Series(data)  
ser
```

```
0    4  
1    7  
2   -5  
3    3  
dtype: int32
```

3. index 지정

- 기본 인덱스
 - 자동으로 0부터 순차적으로 증가하는 정수 인덱스로 설정됨
- 사용자 정의 인덱스
 - series 객체를 생성할 때 index 항목 설정
 - `series.index` 에 직접 할당

3.

인덱스 설정

```
data = [5, 6, 7, 8]
ser = pd.Series(data, index=list('abcd'))
ser.index = list('abcd')
```

```
a    5
b    6
c    7
d    8
dtype: int64
```

3. 인덱스 리셋하기

- 기본 인덱스 (0부터 증가하는 정수형 인덱스)로 변경 됨

```
ser.reset_index(drop=True)
```

```
a    5  
b    6  
c    7  
d    8  
dtype: int64
```



```
0    5  
1    6  
2    7  
3    8  
dtype: int64
```

3. 인덱스 설정

- Name 속성은 dataframe으로 변환하면 column 이름이 됨

```
data = [5, 6, 7, 8]
ser = pd.Series(data, name='size')
ser.name = 'size'
```

```
0    5
1    6
2    7
3    8
Name: size, dtype: int64
```

Dataframe 객체

4. DataFrame

- DataFrame은 표 형식의 데이터 구조
 - 행과 열로 이루어진 2차원 배열
 - 엑셀 스프레드시트와 유사한 형태로 데이터를 조작할 수 있음
 - 다양한 유형의 데이터를 저장하고 처리하는 데 사용
 - 각 열은 데이터의 특정 유형(숫자, 문자열, 날짜 등)을 나타내는 Series 객체로 구성
- DataFrame의 구조
 - 여러 개의 Series가 열로 결합된 형태
 - 각 열은 서로 다른 데이터 타입 가능
 - 행과 열 모두에 인덱스 존재
- DataFrame vs 엑셀 시트
 - 공통점: 행과 열로 구성된 표 형태
 - 차이점: 프로그래밍을 통한 자동화, 대용량 처리 가능

4. Dataframe

- 표 (스프레드시트) 형식의 자료구조

columns

	total_bill	tip	sex	smoker	day	time	size
1	27.20	4.00	Male	No	Thur	Lunch	4
2	20.27	2.83	Female	No	Thur	Lunch	2
3	16.00	2.00	Male	Yes	Thur	Lunch	2
4	13.16	2.75	Female	No	Thur	Lunch	2
5	16.82	4.00	Male	Yes	Sun	Dinner	2
6	23.17	6.50	Male	Yes	Sun	Dinner	4

index values

4. DataFrame 생성하기

- NumPy array 사용하기

```
df = pd.DataFrame(np.arange(9).reshape(3,3))  
df.columns = list('abc')
```

	a	b	c
<i>0</i>	0	1	2
<i>1</i>	3	4	5
<i>2</i>	6	7	8

4 DataFrame 생성하기

- From Series 연결

```
ser1 = pd.Series([1,2,3,4], name='col1')  
ser2 = pd.Series([5,6,7,8], name='col2')  
df = pd.concat([ser1, ser2], axis=1)
```

	col1	col2
0	1	5
1	2	6
2	3	7
3	4	8

4. DataFrame 생성하기

- From dict

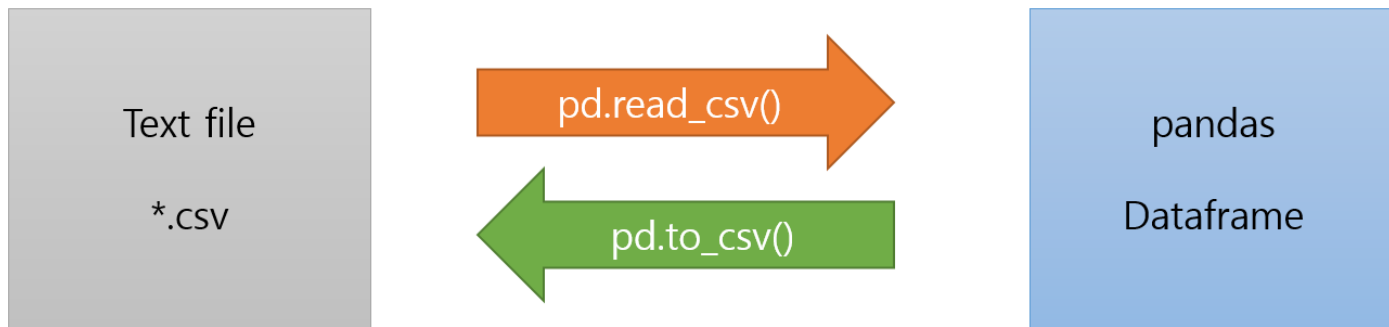
```
d = {"col1": [1, 2, 3, 4], "col2": [5, 6, 7, 8]}  
df = pd.DataFrame(d)
```

	col1	col2
0	1	5
1	2	6
2	3	7
3	4	8

데이터 로딩과 저장

5. pandas 파일 입/출력

- 파일을 읽어서 dataframe을 생성해주고, 파일에 쓰기 위한 함수가 파일 형식별로 존재함
 - 예) CSV



5. 데이터 로딩과 저장

- 파일 형식별 특징
 - CSV: 가장 범용적, 텍스트 형태로 저장
 - Excel: 서식 정보 포함, 여러 시트 지원
 - JSON: 웹 데이터 교환에 적합
 - Pickle: pandas 전용, 빠른 저장/로딩
- 데이터 로딩 시 주의사항
 - 인코딩 문제 (한글 깨짐 현상)
 - 헤더 행 위치 확인
 - 결측치 표현 방식 통일

5. pandas 파일 파싱 함수

- 대표적인 함수들

Function	Description
<code>read_csv</code>	파일, URL 또는 파일류 객체에서 구분된 데이터를 로드 합니다. 쉼표를 기본 구분 기호로 사용
<code>read_excel</code>	Excel XLS 또는 XLSX 파일에서 테이블 형식 데이터 읽기
<code>read_hdf</code>	Read HDF5 files written by pandas
<code>read_html</code>	주어진 HTML 문서에서 찾은 모든 테이블 읽기
<code>read_json</code>	JSON(JavaScript Object Notation) 문자열 표현, 파일, URL 또는 파일류 객체에서 데이터 읽기
<code>read_sas</code>	SAS 시스템의 사용자 지정 스토리지 형식 중 하나에 저장된 SAS 데이터셋을 읽기
<code>read_spss</code>	SPSS에서 생성한 데이터 파일 읽기

5. File로 부터 data 읽고 쓰기

- csv file

```
df = pd.read_csv('filename.csv')  
df.to_csv('output.csv')
```

- excel file

```
df = pd.read_excel('filename.xlsx')    # pip install xlrd  
df.to_excel('output.xlsx')            # pip install openpyxl
```

5. column 이름 직접 지정하기

- example.csv

```
1,2,3,4,hello  
5,6,7,8,world  
9,10,11,12,foo
```

- name 인자를 지정

```
df = pd.read_csv('example.csv', names=['a', 'b', 'c', 'd', 'msg'])
```

	a	b	c	d	msg
0	1	2	3	4	hello
1	5	6	7	8	world
2	9	10	11	12	foo

- name 인자를 지정하지 않은 경우

```
df = pd.read_csv('example.csv')
```

	1	2	3	4	hello
0	5	6	7	8	world
1	9	10	11	12	foo

5. 구분자 지정하기

- example.csv

```
1/2/3/4/hello  
5/6/7/8/world  
9/10/11/12/foo
```

- 구분자 지정하기

```
df = pd.read_table('example.csv', sep='/',  
                   names=['a', 'b', 'c', 'd', 'msg'])
```

	a	b	c	d	msg
0	1	2	3	4	hello
1	5	6	7	8	world
2	9	10	11	12	foo

5. 그 외 옵션

- `index_col`
 - 인덱스 column 지정할 때 사용

```
df = pd.read_csv("example.csv",  
                 names=['a', 'b', 'c', 'd', 'msg'],  
                 index_col='msg')
```

	a	b	c	d
msg				
hello	1	2	3	4
world	5	6	7	8
foo	9	10	11	12

- `nrows`
 - 파일의 첫 부분만 일부 읽어올 때, 몇 줄을 읽을지 지정

```
df = pd.read_csv("example.csv", nrows=5)
```

5. Text 파일에 데이터 쓰기

- `dataframe.to_csv()`
 - 구분자로 분리된 형식으로 데이터를 파일에 씀

	a	b	c	d
msg				
hello	1	2	3	4
world	5	6	7	8
foo	9	10	11	12



```
df.to_csv("out.csv")
```



out.csv

```
msg,a,b,c,d  
hello,1,2,3,4  
world,5,6,7,8  
foo,9,10,11,12
```

5. binary 파일로 저장하기

- 저장하기

```
df.to_pickle('out.pkl')
```

- 저장된 pickle 파일 읽기

```
df = pd.read_pickle('out.pkl')
```

5. MS Excel 파일로 읽기/ 쓰기

- `pd.read_excel(파일이름)`
- `openpyxl`, `xlrd` 패키지를 설치해야 함

```
!pip install xlrd openpyxl
```

```
df = pd.read_excel('test.xlsx')  
df.to_excel('out2.xlsx')
```

5. Seaborn data 읽어오기

- importing seaborn package

```
import seaborn as sns
```

- Dataset 읽어오기

```
df = sns.load_dataset('tips')
```

5. Tips dataset

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4
...	...	...	...	...	...	...	...
239	29.03	5.92	Male	No	Sat	Dinner	3
240	27.18	2.00	Female	Yes	Sat	Dinner	2
241	22.67	2.00	Male	Yes	Sat	Dinner	2
242	17.82	1.75	Male	No	Sat	Dinner	2
243	18.78	3.00	Female	No	Thur	Dinner	2

5. 데이터 셋 설명

- total_bill (총 청구액): 한 그룹의 고객이 식사에 대해 총 청구된 금액
- tip (팁): 식당에서 지불한 팁의 금액
- sex (성별): 고객의 성별
- smoker (흡연 여부): 고객의 흡연 여부
- day (요일): 식당 방문 요일
- time (시간대): 식사 시간대
- size (인원 수): 식사 인원 수

Dataset 탐색을 위한 기본 함수

6. Dataset 준비

```
import seaborn as sns  
df = sns.load_dataset('tips')
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

6. head() 와 tail() 메서드

- `df.head()`
 - 처음 5개 행 출력
- `df.tail()`
 - 마지막 5개 행 출력

6. Descriptive statistics (기술 통계량) 확인

- describe()
 - 수치형 데이터의 분포 파악
 - 숫자형 데이터에 대해서만 적용됨

```
df.describe()
```

	total_bill	tip	size
count	244.000000	244.000000	244.000000
mean	19.785943	2.998279	2.569672
std	8.902412	1.383638	0.951100
min	3.070000	1.000000	1.000000
25%	13.347500	2.000000	2.000000
50%	17.795000	2.900000	2.000000
75%	24.127500	3.562500	3.000000
max	50.810000	10.000000	6.000000

6. summary of DataFrame

- info()
 - 메모리 사용량, 결측치 확인

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 244 entries, 0 to 243
```

```
Data columns (total 8 columns):
```

#	Column	Non-Null Count	Dtype
0	total_bill	244 non-null	float64
1	tip	244 non-null	float64
2	sex	244 non-null	category
3	smoker	244 non-null	category
4	day	244 non-null	category
5	time	244 non-null	category
6	size	244 non-null	int64
7	md	0 non-null	float64

```
dtypes: category(4), float64(3), int64(1)
```

```
memory usage: 9.3 KB
```

6. 범주형 자료

- `unique()`
 - 고유값 종류 파악
 - 범주형 자료 종류 확인

```
df['day'].unique()
```

```
['Sun', 'Sat', 'Thur', 'Fri']
```

```
Categories (4, object): ['Thur', 'Fri', 'Sat', 'Sun']
```

- `value_counts()`
 - 범주형 데이터의 빈도 확인

```
df.value_counts(['smoker'])
```

```
smoker
```

```
No          151
```

```
Yes          93
```

```
Name: count, dtype: int64
```

6.

컬럼 선택

- `dataframe['컬럼 이름']`

```
df['day'], df.smoker  
df[['day', 'smoker']]
```

	0	1	2	3	4	5	6
	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

6. 컬럼 추가 (기존 컬럼으로 부터)

- `dataframe['새 컬럼 이름'] = data`

```
df['ind_bill'] = df['total_bill'] / df['size']
```

또는

```
df.assign(ind_bill=df['total_bill'] / df['size'])
```

6. Column 삭제

```
del dataframe['컬럼 이름']  
del df['ind_bill'] # 추가한 컬럼 삭제  
  
df2.drop('size', axis=1)
```

6. Row 추가

- Append (단일 값으로 마지막에 추가)

```
df.loc[244] = np.nan
```

- List로 추가

```
df.loc[245] = [18.78, 3.00, 'Female', 'No', 'Thur',  
              'Dinner', 2.0]
```

6. Row 삭제

- Row 삭제

Drop ()

```
df.drop(0)
```

```
df.drop([1,2,4])
```

```
df.drop(np.arange(5))
```

```
df.drop(0, inplace=True) # 원본 데이터에 적용!
```

6. row – column 전환

- row – column 전환

```
df.iloc[0:5,:].T
```

	0	1	2	3	4
total_bill	16.99	10.34	21.01	23.68	24.59
tip	1.01	1.66	3.5	3.31	3.61
sex	Female	Male	Male	Male	Female
smoker	No	No	No	No	No
day	Sun	Sun	Sun	Sun	Sun
time	Dinner	Dinner	Dinner	Dinner	Dinner
size	2	3	3	2	4
md	NaN	NaN	NaN	NaN	NaN

데이터 인덱싱과 슬라이싱

7. 슬라이싱 개념

- 슬라이싱(Slicing)의 개념
 - 데이터의 일부분을 연속적으로 선택하는 방법
 - 시작점과 끝점을 지정하여 범위를 선택
 - Python의 리스트 슬라이싱과 유사한 개념
- 기본 문법: [시작:끝:간격]
 - 시작: 슬라이싱을 시작할 위치 (포함)
 - 끝: 슬라이싱을 끝낼 위치 (제외)
 - 간격: 선택할 요소들 사이의 간격 (기본값: 1)

7. pandas에서의 슬라이싱 특징

- pandas 슬라이싱의 특징
 - 행(row) 슬라이싱과 열(column) 슬라이싱 모두 가능
 - 정수 인덱스와 라벨 인덱스 모두 지원
 - 조건부 슬라이싱으로 필터링과 결합 가능

```
df[1:4]      # 1번째부터 3번째 행까지 선택
df[:3]       # 처음부터 2번째 행까지 선택
df[2:]       # 2번째 행부터 끝까지 선택
df[:, :2]    # 모든 행을 2개씩 건너뛰며 선택
```

7. 행 슬라이싱 vs 열 슬라이싱

- 행(Row) 슬라이싱
 - DataFrame의 여러 행을 연속적으로 선택
 - 기본 슬라이싱은 행을 대상으로 함

```
df[1:4] # 1, 2, 3번째 행 선택  
df['2023-01-01':'2023-01-03'] # 날짜 범위로 행 선택
```

- 열(Column) 슬라이싱
 - 여러 열을 연속적으로 선택
 - loc, iloc과 함께 사용

```
df.loc[:, 'tip':'size'] # tip부터 size까지 모든 열  
df.iloc[:, 1:4] # 1번째부터 3번째 열까지
```

7. 인덱싱 vs 슬라이싱

- 인덱싱 vs 슬라이싱
 - 인덱싱: 특정 위치의 단일 요소 선택
 - 슬라이싱: 연속된 범위의 여러 요소 선택

인덱싱 (단일 선택)

`df.loc[0]`

0번째 행만 선택 → Series 반환

`df['tip']`

tip 열만 선택 → Series 반환

슬라이싱 (범위 선택)

`df.loc[0:2]`

0~2번째 행 선택 → DataFrame

반환

`df[['tip', 'size']]`

여러 열 선택 → DataFrame 반환

7. 슬라이싱 시 주의사항

- 슬라이싱 주의사항
 - 정수 인덱스 슬라이싱: 끝 인덱스 제외 (`df[1:4]` → 1,2,3)
 - 라벨 인덱스 슬라이싱: 끝 라벨 포함 (`df.loc['A':'C']` → A,B,C)
 - 빈 결과 처리: 조건에 맞는 데이터가 없을 때 빈 DataFrame 반환
- 성능 고려사항
 - 슬라이싱은 원본 데이터의 뷰(view)를 생성
 - 복사본이 필요한 경우 `.copy()` 메서드 사용
 - 대용량 데이터에서는 필요한 부분만 슬라이싱하여 메모리 절약

7. loc와 iloc의 기본 개념

- loc: 라벨(Label) 기반 인덱싱
 - 행과 열의 이름(라벨)을 사용하여 데이터 선택
 - 인덱스와 컬럼명으로 데이터에 접근
- loc: 정수 위치(Integer Location) 기반 인덱싱
 - 행과 열의 숫자 위치를 사용하여 데이터 선택
 - 0부터 시작하는 정수 인덱스로 데이터에 접근
- 기억하기 쉬운 방법
 - loc = Location by Label (라벨로 위치 찾기)
 - iloc = Integer Location (정수로 위치 찾기)

7. 슬라이싱에서의 중요한 차이점

- loc 슬라이싱: 끝 인덱스 포함 (inclusive)
 - `df.loc[0:3]` → 인덱스 0, 1, 2, 3 모두 포함
- iloc 슬라이싱: 끝 인덱스 제외 (exclusive)
 - `df.iloc[0:3]` → 인덱스 0, 1, 2만 포함 (3은 제외)

<code>df.loc[1:3, 'tip']</code>	# 인덱스 1, 2, 3의 tip 값
<code>df.iloc[1:3, 1]</code>	# 1, 2번째 행의 1번째 열 값

7. 사용 예시

- loc 사용 예시 (라벨 기반)
 - 단일 행 선택: `df.loc[0]`
 - 여러 행 선택: `df.loc[0:2]`
 - 특정 컬럼 선택: `df.loc[:, 'tip']`
 - 행과 열 동시 선택: `df.loc[0:2, 'tip':'size']`
 - 조건부 선택: `df.loc[df['tip'] > 3, 'sex']`
- iloc 사용 예시 (위치 기반)
 - 단일 행 선택: `df.iloc[0]`
 - 여러 행 선택: `df.iloc[0:3]`
 - 특정 컬럼 선택: `df.iloc[:, 1]`
 - 행과 열 동시 선택: `df.iloc[0:3, 1:4]`
 - 간격을 두고 선택: `df.iloc[::2, ::2]`

7. 언제 어떤 것을 사용할지 가이드

- loc를 사용하는 경우
 - 의미 있는 인덱스나 컬럼명이 있을 때
 - 조건부 필터링과 함께 사용할 때
 - 날짜나 문자열 인덱스를 다룰 때
 - 데이터의 의미를 중시하는 분석 시
- iloc를 사용하는 경우
 - 데이터의 위치만 중요할 때
 - 처음 몇 개, 마지막 몇 개 데이터가 필요할 때
 - 규칙적인 간격으로 데이터를 선택할 때
 - 인덱스나 컬럼명을 모르는 상황에서

7. 자주 발생하는 오류들

- **KeyError: loc에서 존재하지 않는 라벨 사용**
 - 해결: 인덱스와 컬럼명을 먼저 확인
- **IndexError: iloc에서 범위를 벗어난 위치 접근**
 - 해결: 데이터프레임의 크기를 먼저 확인
- **혼용 오류: loc에 정수만 사용하거나 iloc에 라벨 사용**
 - 해결: 각각의 목적에 맞게 일관성 있게 사용
- **디버깅 팁**
 - df.shape로 데이터 크기 확인
 - df.index와 df.columns로 라벨 확인
 - 작은 범위부터 테스트해보기

7. 데이터 인덱싱, 슬라이싱

- Indexing and selection

Operation	Syntax	Result
Select column	<code>df[col]</code>	Series
Select row by label (index)	<code>df.loc[index]</code>	Series
Select row by integer location	<code>df.iloc[loc]</code>	Series
Slice rows	<code>df[5:10]</code>	DataFrame
Select rows by boolean vector	<code>df[bool_vec]</code>	DataFrame

데이터 필터링

8. 데이터 필터링

- 실제 분석 시나리오
 - 특정 기간 데이터 추출
 - 특정 조건을 만족하는 고객 찾기
 - 이상치 제거
- 성능 최적화
 - 인덱스 활용한 빠른 필터링
 - 메모리 효율적인 조건문 작성
 - 대용량 데이터 처리 시 주의사항

8. 조건에 맞는 행 선택: 필터링

- 조건을 활용한 필터링

- 조건 연산자 (==, >, <, >=, <=, !=)를 사용
- 단일 조건: `df[df['column'] > value]`

```
df[df['tip'] >= 3] # tip이 3달러 이상인 행 선택  
df[df['tip'] <= 3] # tip이 3달러 이하인 행 선택
```

- 여러 조건의 결합

- 논리 연산자 (&, |)를 사용하여 여러 조건을 결합
- 복합 조건: `df[(조건1) & (조건2)]`
- 주의: and/or 대신 &| 사용

```
df[(df['tip'] > 3) & (df['sex'] == 'Male')]  
# tip이 3달러 초과이고 성별이 남성인 행만 선택
```

8. 다양한 함수 활용

- `isin()`
 - 개념: 특정 값들이 포함된 행을 선택하는 함수
 - 사용법: `df[df['컬럼명'].isin([값1, 값2, ...])]`

```
df[df['day'].isin(['Sat', 'Sun'])]  
# tip이 2~5달러 사이인 행만 선택
```

- `between()`
 - 개념: 특정 범위 내의 값을 가진 행을 선택하는 함수
 - 사용법: `df[df['컬럼명'].between(최솟값, 최댓값)]`

```
df[df['tip'].between(2, 5)]  
# tip이 2~5달러 사이인 행만 선택
```

- `str.contains()`
 - 개념: 문자열 컬럼에서 특정 문자나 패턴을 포함한 행을 선택
 - 사용법: `df[df['컬럼명'].str.contains('검색할문자')]`

```
df[df['day'].str.contains('S', na = False)]  
# day 컬럼에 'S'가 포함된 요일만 선택
```

8. 결측치 필터링

- 결측치(NaN)인 행만 선택
 - 함수: `isna()` 또는 `isnull()`
 - 사용법: `df[df['컬럼명'].isna()]`

```
df[df['tip'].isna()] # tip이 결측치(NaN)인 행만 선택
```

- 결측치가 아닌 행만 선택
 - 함수: `notna()` 또는 `notnull()`
 - 사용법: `df[df['컬럼명'].notna()]`

```
df[df['tip'].notna()] # tip이 결측치가 아닌 행만 선택
```

8. 다양한 방법으로 필터링

- query() 메서드 사용
 - 개념: SQL과 유사한 문법으로 데이터 필터링
 - 장점:
 - 직관적이고 읽기 쉬운 문법
 - 복잡한 조건을 간단하게 표현
 - 문자열 형태로 조건 작성 가능
 - 사용법: df.query("조건문")

```
df.query("tip > 3 and sex == 'Female'")  
# tip이 3초과이고, 성별이 여성인 행만 선택
```

8. `iloc`, `loc`를 활용한 행/열 선택

- `iloc` (정수 위치 기반 선택)
 - 0부터 시작하는 정수 인덱스 사용
 - 행과 열의 위치로 데이터 선택

```
df.iloc[0:3] # 0~2번째 행 선택
```

- `loc` (라벨 기반 선택)
 - 인덱스와 컬럼명으로 데이터 선택

```
df.loc[0:3, 'tip'] # 인덱스 0~3번의 tip 컬럼만 선택
```

- 조건과 결합
 - 필터링과 동시에 특정 컬럼만 선택

```
df.loc[df['tip'] > 3, ['sex', 'tip']]  
# tip이 3초과인 행의 sex, tip 컬럼만 선택
```

8. 필터링 후 행/값 선택 및 변경

- 필터링 후 행/값 선택 및 변경

```
# tip이 3초과인 첫 번째 행 전체
row1 = df[df['tip'] > 3].iloc[0]

# tip이 3초과인 인덱스 5번의 성별
sex5 = df[df['tip'] > 3].loc[5, 'sex']

# tip이 2보다 작은 값은 모두 2로 변경
df.loc[df['tip'] < 2, 'tip'] = 2

# 여성이고 tip이 5초과면 smoker를 'Yes'로 변경
df.loc[(df['sex'] == 'Female') & (df['tip'] > 5),
'smoker'] = 'Yes'

# iloc를 사용해 0번째 행, 1번째 열(tip)을 10으로 변경
df.iloc[0, 1] = 10
```

9.

추출

- 데이터프레임의 전체 데이터 중 일부를 추출

```
df.sample(n=10)
```

- 전체 데이터에서 추출할 비율을 지정하여 샘플링

```
df.sample(frac=0.05)
```

8. 다양한 방법으로 필터링

- 사용자 정의 함수 적용
 - 개념: 복잡한 조건을 함수로 정의하여 적용
 - 방법
 - 조건을 판단하는 함수 정의
 - apply() 메서드로 각 행에 함수 적용
 - 반환된 boolean 값으로 필터링

```
def high_tip(row):  
    return row['tip'] > 5 and row['size'] >= 4  
  
df[df.apply(high_tip, axis=1)]  
# tip이 5초과이고 인원수가 4명 이상인 행만 선택
```

8. 다양한 방법으로 필터링

- `apply()` 함수란?
 - DataFrame이나 Series에 사용자 정의 함수를 적용하는 강력한 도구
 - 벡터화된 연산으로 반복문 없이 전체 데이터에 함수 적용
 - 복잡한 데이터 변환과 계산을 효율적으로 수행
- 기본 문법
 - `df.apply(function, axis=0/1)`
 - `axis=0`: 각 열(column)에 함수 적용
 - `axis=1`: 각 행(row)에 함수 적용

8. 다양한 방법으로 필터링

- 함수별 특징과 사용 시기
 - `apply()`: DataFrame 전체 또는 Series에 함수 적용
 - `map()`: Series의 각 요소에 함수 적용 (값 매핑에 특화)
 - `applymap()`: DataFrame의 모든 요소에 함수 적용
- 성능 고려사항
 - 벡터화된 pandas 함수가 있다면 우선 사용
 - `apply()`는 복잡한 로직이 필요할 때 활용
 - 대용량 데이터에서는 성능 최적화 고려

8. 다양한 방법으로 필터링

단일 컬럼에 함수 적용

```
df['tip_category'] = df['tip'].apply(lambda x: 'High' if  
x > 3 else 'Low')
```

여러 컬럼을 활용한 복잡한 계산

```
def calculate_tip_ratio(row):  
    return row['tip'] / row['total_bill'] * 100
```

```
df['tip_ratio'] = df.apply(calculate_tip_ratio, axis=1)
```

조건부 로직 적용

```
def categorize_customer(row):  
    if row['total_bill'] > 30 and row['tip'] > 5:  
        return 'VIP'  
    elif row['total_bill'] > 20:  
        return 'Regular'  
    else:  
        return 'Basic'
```

```
df['customer_type'] = df.apply(categorize_customer, axis=1)
```

8. 다양한 방법으로 필터링

- lambda 함수 활용
 - 간단한 함수를 한 줄로 정의
 - apply()와 함께 사용하여 코드 간소화
 - 복잡한 로직은 일반 함수로 정의 권장

```
# 기본 lambda 사용
```

```
df['total_bill_rounded'] =  
df['total_bill'].apply(lambda x: round(x, 1))
```

```
# 조건부 lambda 사용
```

```
df['meal_time'] = df['time'].apply(lambda x: '점심'  
if x == 'Lunch' else '저녁')
```

데이터 정렬

9. 데이터 정렬

- 데이터 정렬이란?
 - 데이터를 특정 기준에 따라 순서대로 배열하는 과정
 - 데이터 분석의 기본이 되는 중요한 전처리 작업
- 정렬의 활용 사례
 - 상위/하위 데이터 확인 (매출 순위, 성적 순위 등)
 - 데이터 패턴 파악 및 이상치 발견
 - 시계열 데이터의 시간순 정렬
 - 그룹별 분석을 위한 사전 정렬

9.

단일 컬럼 정렬

- 단일 컬럼 정렬이란?
 - 하나의 컬럼을 기준으로 전체 데이터를 정렬하는 방법
 - 가장 기본적이고 자주 사용되는 정렬 방식
 - 숫자형과 문자형 데이터 모두 정렬 가능
- `sort_values()` 기본 문법
 - `df.sort_values(by='컬럼명')`
 - `by`: 정렬 기준이 될 컬럼명 지정
 - `ascending`: 정렬 방향 (`True`=오름차순, `False`=내림차순)

```
df.sort_values(by='tip') # 기본 오름차순 정렬
```

```
df.sort_values(by='tip', ascending=False) # 내림차순 정렬
```

```
df.sort_values(by='day') # 문자형 컬럼 정렬 (알파벳 순)
```

```
# 정렬 결과를 원본에 적용
```

```
df.sort_values(by='tip', inplace=True)
```

9. 다중 컬럼 정렬

- 다중 컬럼 정렬의 우선순위
 - 첫 번째 컬럼으로 1차 정렬, 두 번째 컬럼으로 2차 정렬
 - 첫 번째 컬럼 값이 같을 때만 두 번째 컬럼으로 정렬 수행
 - 각 컬럼마다 다른 정렬 방향 지정 가능
- 다중 정렬의 활용
 - 데이터를 체계적으로 분류하고 정리
 - 우선순위가 있는 정렬 조건 적용
 - 그룹별 분석을 위한 사전 정렬

```
df.sort_values(by=['smoker', 'tip']) # 기본 다중 컬럼 정렬 (모두 오름차순)
```

```
# 각 컬럼마다 다른 정렬 방향: smoker는 오름차순, tip은 내림차순으로 정렬
```

```
df.sort_values(by=['smoker', 'tip'],  
               ascending=[True, False])
```

```
# 3개 컬럼 정렬
```

```
df.sort_values(by=['day', 'smoker', 'tip'],  
               ascending=[True, True, False])
```

9. 결측치 데이터 정렬

- 결측치 처리 옵션
 - na_position 매개변수로 결측치 위치 조정
 - 'first': 결측치를 맨 앞에 배치
 - 'last': 결측치를 맨 뒤에 배치 (기본값)

```
df.sort_values(by='tip', na_position='first')
```

```
# 결측치를 맨 앞에 두고 정렬
```

```
df.sort_values(by='tip', na_position='last')
```

```
# 결측치를 맨 뒤에 두고 정렬
```

9. 사용자 정의 정렬

- key 매개변수를 활용한 사용자 정의 정렬
 - 복잡한 정렬 기준 적용 가능
 - 함수를 통한 변환 후 정렬

```
# 문자열 길이 기준 정렬
```

```
df.sort_values(by='day', key=lambda x: x.str.len())
```

```
# 절댓값 기준 정렬
```

```
df.sort_values(by='tip', key=lambda x: x.abs())
```

```
# 사용자 정의 순서로 정렬 (요일 순서)
```

```
day_order = {'Mon': 1, 'Tue': 2, 'Wed': 3, 'Thu': 4,  
             'Fri': 5, 'Sat': 6, 'Sun': 7}
```

```
df.sort_values(by='day', key=lambda x: x.map(day_order))
```

9. 범주형 데이터 정렬

- 범주형 데이터의 순서 지정
 - Categorical 데이터 타입 활용
 - 논리적 순서에 따른 정렬

범주형 데이터로 변환 후 정렬

```
df['day'] = pd.Categorical(df['day'],  
                           categories=['Thur', 'Fri', 'Sat', 'Sun'],  
                           ordered=True)
```

```
df.sort_values(by='day')
```

크기 순서 지정 (Small, Medium, Large)

```
size_order = pd.CategoricalDtype(['Small', 'Medium', 'Large'],  
                                  ordered=True)
```

```
df['size_category'] = df['size_category'].astype(size_order)
```

9. 정렬 후 순위 매기기

- 정렬 후 순위 매기기
 - rank() 메서드와 정렬의 조합
 - 동점자 처리 방법

```
df_sorted = df.sort_values(by='tip', ascending=False)
df_sorted['tip_rank'] = df_sorted['tip'].rank(ascending=False)
```

- 상위/하위 N개 데이터 추출
 - nlargest(), nsmallest() 메서드 활용
 - 정렬보다 효율적인 방법

```
df.nlargest(5, 'tip') # 팁이 가장 큰 5개 행
df.nsmallest(3, 'total_bill') # 총 청구액이 가장 작은 3개 행
```

데이터 정제

10. 데이터 정제 및 준비

- 주어진 데이터를 구조화하고 분석에 적합하도록 준비하는 과정
 - 데이터 불러오기
 - 데이터 정제하기
 - 데이터 변형하기
 - 데이터 재정렬
- 목적: 데이터를 원하는 형태로 가공하는 것!

10. 데이터 준비/정제 관련 주제

- 결측 데이터 처리 (Handling missing data)
- 데이터 변형 (Data transformation)

10. 결측치 처리

- 수집된 데이터에 누락된 값이 존재할 경우
 - 해당 데이터를 삭제
 - 다른 값으로 채움
- `isna()`
- `fillna()`
- `dropna()`

10. 결측치 확인

- 결측치는 numpy 의 nan 상수로 처리됨

```
df['md'] = np.nan      # 임의로 결측치 생성
df.isna()              # 각 셀 값이 NaN인지 T/F로 반환
df.isna().sum()        # 각 셀에서 결측치가 몇 개 인지 확인
```

10. 결측 데이터 제외하기 : dropna()

```
data = pd.DataFrame([[1., 6.5, 3.],  
                    [1., np.nan, np.nan],  
                    [np.nan, np.nan, np.nan],  
                    [np.nan, 6.5, 3.]])
```

	0	1	2
0	1.0	6.5	3.0
1	1.0	NaN	NaN
2	NaN	NaN	NaN
3	NaN	6.5	3.0

- dropna()
 - 하나라도 NaN이 포함된 행 제외
 - thresh : 개수 이상인 행만 남김
 - how='all' 로 주면, 모든 컬럼의 값이 nan인 행을 삭제
- Isna()
 - 하나라도 NaN이 포함된 행 제외

10. 결측치 채우기

```
df['column_name'].fillna(50) # 특정값으로 채우기  
df.fillna(method='ffill') # 이전 행의 값으로 채움  
df.fillna({'tip':0., 'sex':'Male'}) # 열마다 다른 값 채우기  
df.fillna(data.mean()) # 각 열 평균값으로 채우기
```

10. 데이터변형

- 중복 제거
- 함수, 맵핑을 이용한 변형
- 값 치환
- 축 색인 이름 변경
- 개별화, 양자화
- 이상치 찾아 제외하기
- 랜덤 샘플링

10. 중복 값 찾기

- 이전에 동일한 row 가 존재하는지 boolean series 로 반환
 - df.duplicated()

```
df.loc[244] = df.loc[243] # 동일한 row  
df.duplicated()
```

...

242 False

243 False

244 True

Length: 245, dtype: bool

10. 중복 값 제거하기

- 중복 값 제거하기

```
df.drop_duplicates()  
df.drop_duplicates(['tip'])    # 245 → 123 rows  
df.drop_duplicates('tip', keep='first')
```

10. map 함수 활용하기

- 한글 표현 열을 하나 추가하려 할 때,

```
d = {1: '하나', 2: '둘', 3: '셋', 4: '넷', 5: '다섯', 6: '여섯'}  
df['kr'] = df['size'].map(lambda x: d[x])
```

	total_bill	tip	sex	smoker	day	time	size	kr
0	16.99	1.01	Female	No	Sun	Dinner	2	둘
1	10.34	1.66	Male	No	Sun	Dinner	3	셋
2	21.01	3.50	Male	No	Sun	Dinner	3	셋
3	23.68	3.31	Male	No	Sun	Dinner	2	둘
4	24.59	3.61	Female	No	Sun	Dinner	4	넷
...	...	...	...	...	...	...	...	...

10. 더미(dummy)행렬 변환하기

- 더미(dummy) 행렬 변환하기

```
pd.get_dummies(df['day']).astype(int)
```

	Fri	Sat	Sun	Thur
0	0	0	1	0
1	0	0	1	0
2	0	0	1	0
3	0	0	1	0
4	0	0	1	0
..	...	...	...	...
240	0	1	0	0
241	0	1	0	0
242	0	1	0	0
243	0	0	0	1
244	0	0	0	1

[245 rows x 4 columns]

10. 연속 데이터 이산화 (binning)

- 연속 데이터 → categorical (범주형) 데이터

```
pd.cut(df['tip'],  
       bins=[0, 2.5, 5, 7.5, 10])
```

- 레이블을 아래와 같이 주면 구분이 쉬움

```
labels=['Low', 'Mid-Low', 'Mid-High',  
        'High']
```

```
0          Low  
1          Low  
2       Mid-Low  
3       Mid-Low  
4       Mid-Low  
  
...  
239     Mid-High  
240          Low  
241          Low  
242          Low  
243     Mid-Low  
Name: tip, Length: 244,  
dtype: category  
Categories (4, object):  
['Low' < 'Mid-Low'  
< 'Mid-High' < 'High']
```

10. 벡터화된 문자열 처리 함수

- 벡터화된 문자열 처리

```
df['sex'].str.lower()
```

```
0      female
1       male
2       male
3       male
4      female
```

```
...
```

```
239     male
240    female
241     male
242     male
243    female
```

```
Name: sex, Length: 244, dtype: object
```

10. 데이터프레임 연산

- 벡터화 된 다양한 연산 적용 가능

```
df['total_bill'] - 1  
df['total_bill'] * 0.9  
df['tip'] / df['total_bill']
```

10. 값 치환

- 데이터프레임의 값을 일괄적으로 변경
 - inplace=True 인 경우 원본 데이터에 적용됨

```
df['sex'].replace(['Male', 'Female'], ['M', 'F'])
```

데이터 표준화

11. 데이터 표준화

- **표준화(standardization)란?**
 - 수집 & 정리된 데이터의 동일 포맷으로 작업하는 것
 - 데이터 분석 시 정확도를 높임
- **방법**
 - 단위 환산
 - 수집 데이터의 나라별 사용 단위 상이 등
 - 단위별 변환 방식으로 환산
 - 환산된 데이터를 새로운 열(column)에 추가
 - 자료형 변환
 - 기본 자료형
 - 데이터 자료형: 범주형, 연속형

11. 데이터 정규화

- **정규화(Normalization)란?**
 - 각 열에 속하는 숫자 데이터의 상대적 크기 차이를 해결하는 방법
 - 동일한 크기 기준으로 나눈 비율로 나타내는 것
 - 데이터 범위: 0 ~ 1 또는 -1 ~ 1
- **방법**
 - Min-Max 정규화 (Min-Max Normalization)
 - Z-Score 정규화 (Standardization)
 - Max-Abs 정규화 (Max-Abs Scaling)
 - Robust 정규화 (Robust Scaling)

11. 데이터 정규화

- **Min-Max 정규화 (Min-Max Normalization)**

- 데이터의 최소값을 0, 최대값을 1로 변환해 모든 데이터가 0과 1사이에 있도록 조정

- $$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

```
# Min-Max 정규화 함수
```

```
def min_max_normalize(column):
```

```
    return (column - column.min()) / (column.max() - column.min())
```

```
# total_bill 열 정규화
```

```
df['total_bill_min_max_1'] = min_max_normalize(df['total_bill'])
```

```
df['total_bill_min_max_2'] = df[['total_bill']].apply(min_max_normalize)
```

```
# 결과 출력
```

```
print(df[['total_bill', 'total_bill_min_max_1', 'total_bill_min_max_2']].head())
```

	total_bill	total_bill_min_max_1	total_bill_min_max_2
0	16.99	0.291579	0.291579
1	10.34	0.152283	0.152283
2	21.01	0.375786	0.375786
3	23.68	0.431713	0.431713
4	24.59	0.450775	0.450775

11. 데이터 정규화

- **Z-Score 정규화 (Standardization)**

- 데이터의 평균을 0, 표준 편차를 1로 변환해 조정
- $Z = (x - \mu) / \sigma$

```
# Z-Score 정규화 함수
def z_score_normalize(column):
    return (column - column.mean()) / column.std()

# total_bill 열 정규화
df['total_bill_z_score_1'] = z_score_normalize(df['total_bill'])
df['total_bill_z_score_2'] = df[['total_bill']].apply(z_score_normalize)

print(df[['total_bill', 'total_bill_z_score_1', 'total_bill_z_score_2']].head())
```

	total_bill	total_bill_z_score_1	total_bill_z_score_2
0	16.99	-0.314066	-0.314066
1	10.34	-1.061054	-1.061054
2	21.01	0.137497	0.137497
3	23.68	0.437416	0.437416
4	24.59	0.539635	0.539635

11. 데이터 정규화

- **Max-Abs 정규화 (Max-Abs Scaling)**

- 각 데이터 포인트를 해당 절대값의 최대값으로 나눠 $[-1, 1]$ 범위로 조정
- $x' = x/|x_{max}|$

```
# Max-Abs 정규화 함수
```

```
def max_abs_normalize(column):  
    return column / column.abs().max()
```

```
# total_bill 열 정규화
```

```
df['total_bill_max_abs_1'] = max_abs_normalize(df['total_bill'])  
df['total_bill_max_abs_2'] = df[['total_bill']].apply(max_abs_normalize)  
  
print(df[['total_bill', 'total_bill_max_abs_1', 'total_bill_max_abs_2']].head())
```

	total_bill	total_bill_max_abs_1	total_bill_max_abs_2
0	16.99	0.334383	0.334383
1	10.34	0.203503	0.203503
2	21.01	0.413501	0.413501
3	23.68	0.466050	0.466050
4	24.59	0.483960	0.483960

11. 데이터 정규화

• Robust 정규화 (Robust Scaling)

- 중앙값(median)과 IQR(interquartile range)을 사용해 정규화 함.
- 이상치의 영향을 줄이는데 유용
- $x' = x - \text{median} / \text{IQR}$

Robust 정규화 함수

```
def robust_normalize(column):  
    median = column.median()  
    iqr = column.quantile(0.75) - column.quantile(0.25)  
    return (column - median) / iqr
```

total_bill 열 정규화

```
df['total_bill_robust_1'] = robust_normalize(df['total_bill'])  
df['total_bill_robust_2'] = df[['total_bill']].apply(robust_normalize)  
  
print(df[['total_bill', 'total_bill_robust_1', 'total_bill_robust_2']].head())
```

	total_bill	total_bill_robust_1	total_bill_robust_2
0	16.99	-0.074675	-0.074675
1	10.34	-0.691558	-0.691558
2	21.01	0.298237	0.298237
3	23.68	0.545918	0.545918
4	24.59	0.630334	0.630334

시계열 데이터 조작

12. 시계열 데이터

- 시계열 데이터

- 주식, 환율 등 금융 데이터에서 행 인덱스로 자주 사용
- 시간으로 기록된 데이터
- 자료형
 - 특정 시점을 기록 하는 데이터 자료형 → Timestamp
 - 두 시점 사이의 일정한 기간을 나타내는 자료형 → Period

12. 날짜/시간 데이터 변환

• 날짜/시간 데이터 변환

- Timestamp는 연도, 월, 일, 등을 포함한 구체적인 시각을 나타냄
 - Timestamp object = `pandas.to_datetime(1개 데이터)`
 - Datetime Index object = `pandas.to_datetime(복수 데이터)`

[시간 데이터]
문자열 "2025년 7월 22일"
숫자 20220104

Pandas.to_datetime()



Timestamp
객체
datetime64

- `Object.to_datetime(format=str, errors='raise')`

- Error

- Raise : 오류 발생
- Coerce: 오류 미발생
누락값(NA)
- Ignore: 오류 미발생
입력값 반환

옵션	의미	옵션	의미
%Y	4자리 연도	%H	24시간 00~23
%y	2자리 연도	%I	12시간 00~12
%m	0으로 시작 month 01~12	%p	AM. / PM.
%d	0으로 시작 day 0~31	%M	분 00~59
%w	0으로 시작하는 요일 0~6	%s	초 00~59
%A	요일 (Sunday)	%Z	Time zone 이름 UTC, GMT
%a	요일 줄임말 (Sun)	%z	UTC 오프셋
%B	월 (January)	%U	1년 일요일 기준 주일수 (00~53)
%b	월 줄임말 (Jan)	%W	1년 월요일 기준 주일수 (00~53)

12. 다양한 방법으로 필터링

- 문자열을 날짜형으로 변환

```
date_string = '2023-07-13 09:00'  
date = pd.to_datetime(date_string)  
print(date)  
# 문자열을 datetime으로 변환
```

- 데이터프레임 컬럼 변환

```
df = pd.DataFrame({'date': ['2023-07-13', '2023-07-14', '2023-07-15']})  
df['date'] = pd.to_datetime(df['date'])  
print(df)
```

▶ 문자열 형태의 날짜를 datetime 타입으로 변환하면 날짜 연산, 인덱싱 등이 쉬워집니다

12. 날짜 범위 생성

- **Pandas.date_range**

- 연속된 날짜/시간 범위를 생성하는 함수
- DatetimeIndex 객체를 반환하여 시계열 데이터의 인덱스로 활용

```
pandas.date_range(start = '날짜 범위 시작', # 2025-01-01
                  end = '날짜 범위 끝',      # None 범위가 지정 안함
                  periods = '생성 개수',      # 생성할 날짜 개수
                  freq = '시간 간격',         # 시간대 설정
                  tz = 'Asia/Seoul')
```

12. 다양한 방법으로 필터링

- 일정 기간의 날짜 생성

```
dates = pd.date_range(start='2025-07-01',  
                       end='2025-07-07')  
  
print(dates)  
# 2025년 7월 1일부터 7월 7일까지의 날짜 생성
```

- 특정 주기/빈도로 날짜 생성

```
monthly = pd.date_range(start='2025-01-01',  
                        periods=6,  
                        freq='MS')  
  
print(monthly)  
  
# 2023년 1월부터 6개월 동안 월 단위로 날짜 생성
```

12. 시계열 데이터프레임 만들기 및 인덱스 지정

- 시계열 데이터 인덱스 지정
 - 인덱스를 날짜로 지정하면 시계열 분석 및 시각화에 유용합니다.

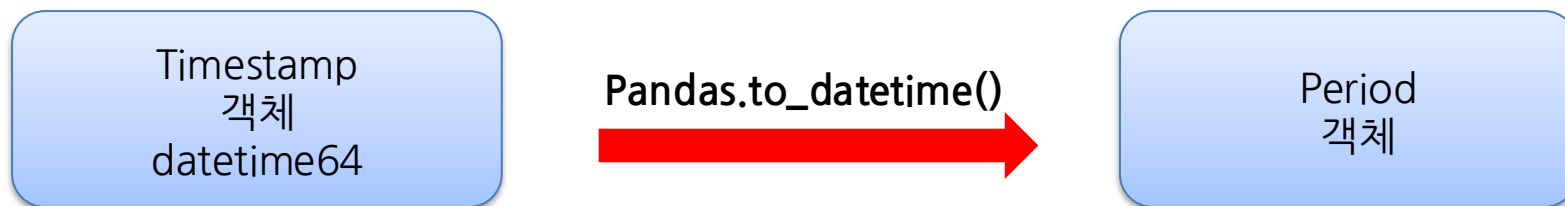
임의의 시계열 데이터 생성

```
dates = pd.date_range('2023-07-01', periods=7)
data = np.random.randint(10, 100, size=7)
df = pd.DataFrame({'value': data}, index=dates)
print(df)
```

12. 시간의 구간이나 범위 변환

• 날짜/시간 데이터 변환

- Period는 시간의 구간이나 범위를 표현
 - Period object = `pandas.to_period()`
 - Periodindex Index object = `pandas.to_period()`



- `Object.to_period(freq = '시간간격')`

옵션	의미	옵션	의미
D	Day	H	Hour
W	Week	T, min	Minute
M	Month end	S	Second
MS	Month begin	L, ms	Millisecond
A, Y	Year end	U, us	Microsecond
AS, YS	Year begin	N	Nanosecond

12. Period 객체 생성

- Period 객체 생성

```
# 2024년 7월 한 달을 나타내는 Period 객체 생성
p = pd.Period('2024-07', freq='M')
print(p)
print("시작일:", p.start_time)
print("종료일:", p.end_time)
```

12. 연속된 범위 생성

- **pandas.period_range()**
 - 연속된 기간(Period) 범위를 생성하는 함수
 - `date_range()`가 특정 시점(Timestamp)들을 생성하는 것과 달리, 이 함수는 시간의 구간이나 범위를 나타내는 Period 객체들을 생성

```
pandas.period_range( start='날짜 범위 시작', # 2020-01-01
                     end='날짜 범위 끝',      # None 범위지정 안 함
                     periods=생성 개수,        # 생성할 Timestamp 개수
                     freq='시간 간격',
                     tz='타임존')
```

12. 시간의 구간이나 범위 활용

- PeriodIndex의 속성 활용

```
# 각 기간의 연도와 월 추출
df['year'] = df.index.year
df['month'] = df.index.month
print(df)
```

- period_range + DataFrame 실습

```
# 분기별(3개월 단위) 기간 생성
pr_q = pd.period_range(start='2023Q1', periods=4, freq='Q')
print(pr_q)

# 주간(Week) 단위 기간 생성
pr_w = pd.period_range(start='2024-07-01', periods=4,
                        freq='W')
print(pr_w)
```

12. 날짜 인덱스 활용

- 날짜로 슬라이싱

```
# 2023년 7월 3일 이후 데이터만 추출  
print(df['2023-07-03:'])
```

- 날짜 속성 활용

```
# 요일 정보 추가  
df['weekday'] = df.index.day_name()  
print(df)
```

- 시계열 데이터 리샘플링

```
# 일별 데이터를 월별 평균으로 리샘플링  
numeric_cols = df_ts.select_dtypes(include=['number']).columns  
monthly_mean = df_ts[numeric_cols].resample('M').mean()  
monthly_mean
```

Thank you

Q & A

